lecture_01.py

```
import os

import regex
from abc import ABC
from dataclasses import dataclass
from collections import defaultdict
from edtrace import link, text, image
from lecture_util import article_link, post_link, video_link, get_local_url
from references import shannon_1950, lstm_1997, brants_2007, bengio_2003, glorot_2010, seq2seq_2014
from references import bahdanau_2015_attention, transformer_2017, gpt2_2019, t5_2019, kaplan_scaling_laws_2020, mup_2022
from references import dpo_2023, adamw_2017, adam_2014, grpo, ppo_2017, muon_2024
from references import large_batch_training_2018, wsd_2024, cosine_learning_rate_2017, moe_2017,
switch_transformers_2021, auxfree_2024, mtp_2024
from references import megatron_lm_2019, shazeer_2020, elmo_2018, bert_2018
from references import rms_norm_2019, layernorm_2016, pre_post_norm_2020, qk_norm_2023
from references import rope_2021, soap_2024, sparse_transformer_2019, gqa_2023, mla_2024
from references import linear_attention_2020, mamba_2_2024, gdn_2024, mamba_3_2026
from references import megabyte_2023, byt5_2021, blt_2024, tfree_2024, hnet_2025, sennrich_2016, zero_2019, gpipe_2018
from references import regmix_2025, olmix_2026, wrap_2024

from references import gpt_3_2020, gpt_4_2023, instruct_gpt_2022
from references import the_pile_2020, gpt_j_2021, opt_175b_2022, bloom_2022, palm_2022, chinchilla_2022
from references import llama_2023, llama_2_2023, llama_3_2024
from references import mistral_7b_2023, mixtral_2024
from references import deepseek_67b_2024, deepseek_v2_2024, deepseek_v3_2024, deepseek_r1_2025, deepseek_v3_2_2025
from references import qwen_2_5_2024, qwen_3_2025
from references import kimi_1_5_2025, kimi_k2_5_2026
from references import glm_4_5_2025, glm_5_2026
from references import minimax_m2_5_2026
from references import xiaomi_mimo_v2_2026

from references import marin_8b_2025, marin_32b_2025
from references import olmo_7b_2024, olmo_2_2025, olmo_3_2025
from references import nemotron_15b_2024, nemotron_3_2025

import tiktoken

def main():
    welcome()
    why_this_course_exists()
    current_lm_landscape()

    what_is_this_program()

    course_logistics()
    course_syllabus()

    tokenization()              # First unit

        Next time: resource accounting


def welcome():

```

CS336: Language Models From Scratch (Spring 2026)


Percy Liang
Instructor


Tatsunori Hashimoto
Instructor


Marcel Rød
CA


Herman Brunborg
CA


Steven Cao
CA

...bringing you the 3rd offering of CS336.

Lectures from 2nd offering (Spring 2025) are on YouTube.
What's new?

- Same 'from scratch' philosophy
- Prioritize high value-per-time concepts, don't lose the forest for the trees
- More coverage of modern LM ingredients (mixture of experts, long-context, agents)

```
def why_this_course_exists():
```

Why did we make this course?

Problem: researchers are becoming **disconnected** from the underlying technology.

- 2016: researchers implemented and trained their own models.
- 2018: researchers downloaded models (e.g., BERT) and fine-tuned them.
- Today: researchers prompt API models (e.g., GPT/Claude/Gemini).

Moving up levels of abstraction boosts productivity, but

- These abstractions are leaky (in contrast to programming languages or operating systems).
- There is still fundamental research to be done that requires tearing up the stack.

Full understanding of this technology is necessary for **fundamental research**.

Philosophy of this course: **understanding via building**.
But there's one small problem...

The industrialization of language models



Frontier models are really expensive:

- 2023: GPT-4 supposedly cost $100M to train. [article]
- 2025: xAI builds cluster with 230K GPUs for training Grok. [article]

There are no public details on how frontier models are built.
From the GPT-4 technical report [OpenAI+ 2023]:

2 Scope and Limitations of this Technical Report

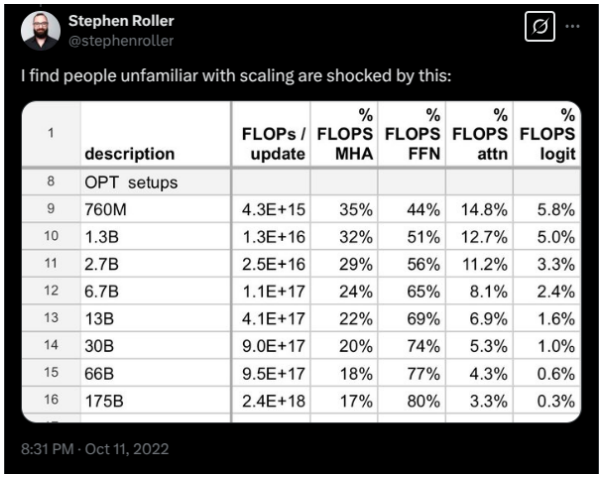
This report focuses on the capabilities, limitations, and safety properties of GPT-4. GPT-4 is a Transformer-style model [39] pre-trained to predict the next token in a document, using both publicly available data (such as internet data) and data licensed from third-party providers. The model was then fine-tuned using Reinforcement Learning from Human Feedback (RLHF) [40]. Given both the competitive landscape and the safety implications of large-scale models like GPT-4, this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar.

We are committed to independent auditing of our technologies, and shared some initial steps and ideas in this area in the system card accompanying this release.[2] We plan to make further technical details available to additional third parties who can advise us on how to weigh the competitive and safety considerations above against the scientific value of further transparency.

Frontier models are out of reach for us.

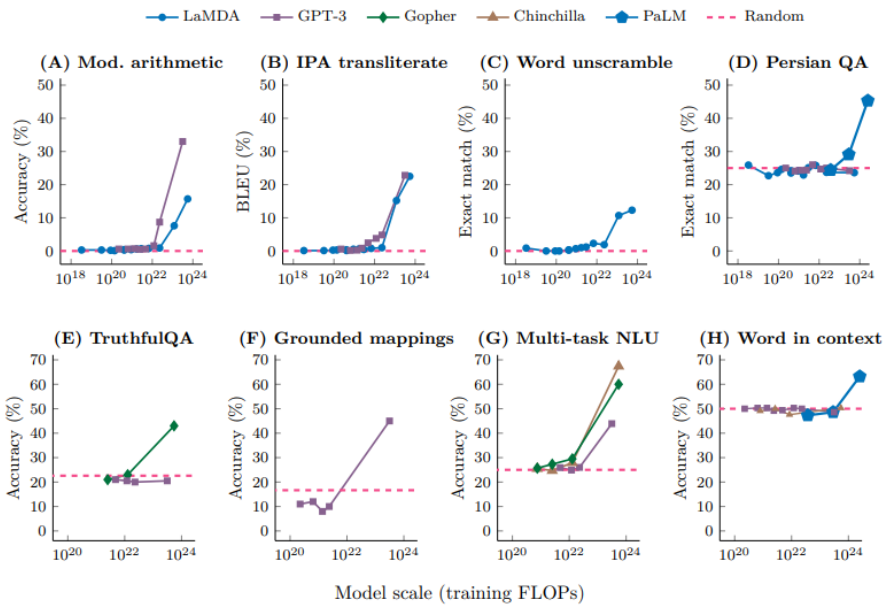
We could build small language models (<1B parameters), but this might not be representative of large language models.

Example 1: fraction of FLOPs spent in attention versus MLP changes with scale. [post]



	description	FLOPs / update	% FLOPS MHA	% FLOPS FFN	% FLOPS attn	% FLOPS logit
8	OPT setups					
9	760M	4.3E+15	35%	44%	14.8%	5.8%
10	1.3B	1.3E+16	32%	51%	12.7%	5.0%
11	2.7B	2.5E+16	29%	56%	11.2%	3.3%
12	6.7B	1.1E+17	24%	65%	8.1%	2.4%
13	13B	4.1E+17	22%	69%	6.9%	1.6%
14	30B	9.0E+17	20%	74%	5.3%	1.0%
15	66B	9.5E+17	18%	77%	4.3%	0.6%
16	175B	2.4E+18	17%	80%	3.3%	0.3%

Example 2: emergence of behavior with scale [Wei+ 2022]



What can we learn in this class that transfers to frontier models?

There are three types of knowledge:

- **Mechanics**: how things work (what a Transformer is, how model parallelism works)
- **Mindset**: squeezing the most out of the hardware, taking scaling seriously
- **Intuitions**: which data and modeling decisions yield good accuracy

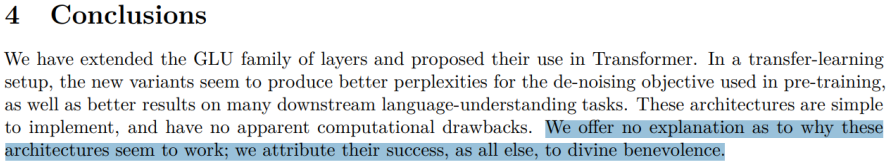
We can teach mechanics and mindset (these do transfer).

We can only partially teach intuitions (do not necessarily transfer across scales).

Intuitions? ??

Some design decisions are simply not (yet) justifiable and just come from experimentation.

Example: Noam Shazeer paper that introduced SwiGLU [Shazeer 2020]



4 Conclusions

We have extended the GLU family of layers and proposed their use in Transformer. In a transfer-learning setup, the new variants seem to produce better perplexities for the de-noising objective used in pre-training, as well as better results on many downstream language-understanding tasks. These architectures are simple to implement, and have no apparent computational drawbacks. We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

The bitter lesson

Wrong interpretation: scale is all that matters, algorithms don't matter.

Right interpretation: algorithms that scale are what matter.

accuracy = efficiency x resources

In fact, efficiency is way more important at larger scales (can't afford to be wasteful).

[Hernandez+ 2020] showed 44x algorithmic efficiency on ImageNet between 2012 and 2019.

Framing: what is the best model one can build given a certain compute and data budget?

In other words, **maximize efficiency**!

```
def current_lm_landscape():
```

Pre-neural (before 2010s)

- Language model to measure the entropy of English [Shannon 1950]
- N-gram language models (used in machine translation and speech recognition systems) [Brants+ 2007]

Neural ingredients (2010s)

- Long-Short Term Memory (LSTM) [Hochreiter+ 1997]
- First neural language model [Bengio+ 2003]
- Sequence-to-sequence modeling (for machine translation) [Sutskever+ 2014]
- Adam optimizer [Kingma+ 2014]
- Attention mechanism (for machine translation) [Bahdanau+ 2014]
- Transformer architecture (for machine translation) [Vaswani+ 2017]
- Mixture of experts [Shazeer+ 2017]
- Model parallelism [Huang+ 2018][Rajbhandari+ 2019][Shoeybi+ 2019]

Early foundation models (late 2010s)

- ELMo: pretraining with LSTMs, fine-tuning improves downstream tasks [Peters+ 2018]
- BERT: pretraining with Transformer, fine-tuning improves downstream tasks [Devlin+ 2018]
- Google's T5 (11B): cast everything as text-to-text [Raffel+ 2019]

Embracing scaling

- OpenAI's GPT-2 (1.5B): fluent text, first signs of zero-shot [Radford+ 2019]
- Scaling laws: provide hope / predictability for scaling [Kaplan+ 2020]
- OpenAI's GPT-3 (175B): in-context learning [Brown+ 2020]
- Google's PaLM (540B): massive scale, undertrained [Chowdhery+ 2022]
- DeepMind's Chinchilla (70B): compute-optimal scaling laws [Hoffmann+ 2022]

Open models

Early attempts (attempts to replicate GPT-3):

- EleutherAI's open datasets (The Pile) and models (GPT-J) [Gao+ 2020][Wang+ 2021]
- Meta's OPT (175B): GPT-3 replication, lots of hardware issues [Zhang+ 2022]
- Hugging Face / BigScience's BLOOM (176B): focused on data sourcing [Workshop+ 2022]

Credible open-weight models (weights + paper):

- Meta's Llama models [Touvron+ 2023] [Touvron+ 2023] [Grattafiori+ 2024]
- Mistral's models [Jiang+ 2023] [Jiang+ 2024]
- DeepSeek's models [DeepSeek-AI+ 2024] [DeepSeek-AI+ 2024] [DeepSeek-AI+ 2024] [DeepSeek-AI+ 2025] [DeepSeek-AI+ 2025]
- Alibaba's Qwen models [Qwen+ 2024] [Yang+ 2025]
- Moonshot's Kimi models [Kimi Team 2025] [Kimi Team 2026]
- Z.ai's GLM models [GLM-4.5 Team 2025] [GLM-5-Team 2026]
- Minimax's models [[MiniMax M2.5]]
- Xiaomi's MIMO models [[Xiaomi MIMO v2]]

These models are approaching closed models (GPT, Claude, Gemini, etc.).

Open-source models (weights + paper + code + data):

- AI2's Olmo models [Groeneveld+ 2024] [Team OLMo 2024] [Team Olmo 2025]
- NVIDIA's Nemotron models [Parmar+ 2024] [NVIDIA+ 2025]
- Marin's models (open development) [[Marin 8B retro]] [[Marin 32B retro]]

Openness is important for trust and innovation [Kapoor+ 2024]
Ideas from open models enable us to teach CS336.

What is a language model?

- 2018 (BERT): something you fine-tune
- 2020 (GPT-3): something you prompt
- 2022 (ChatGPT): something you talk to [example conversation]
- 2026 (agents): something that acts autonomously [example trace]

The fundamentals are the same (attention, kernels, optimization).
The specs are different (longer context, inference efficiency matters even more).

```
def what_is_this_program():
```

This is an *executable lecture*, a program whose execution delivers the content of a lecture.
Executable lectures make it possible to:

- view and run code (since everything is code!),

```
total = 0
for x in [1, 2, 3]:
    total += x
```

- see the hierarchical structure of the lecture

```
def course_logistics():
```

All information online: [course website]

This is a 5-unit class.
Comment from Spring 2024 course evaluation:

> *The entire assignment was approximately the same amount of work as all 5 assignments from CS 224n plus the final project. And that's just the first homework assignment.*

Why you should take this course

- You have an obsessive need to understand how things work.
- You want to build up your research engineering muscles.

Why you should not take this course

- You actually want to get research done this quarter. (Talk to your advisor.)
- You are interested in learning about the hottest new techniques in AI (e.g., multimodality, RAG, etc.). (You

should take a seminar class for that.)
- You want to get good results on your own application domain. (You should just prompt or fine-tune an existing model.)

How you can follow along at home

- All lecture materials and assignments will be posted online, so feel free to follow on your own.
- Lectures are recorded via CGOE.

Assignments

- 5 assignments (basics, systems, scaling laws, data, alignment).
- No scaffolding code, but we provide unit tests and adapter interfaces to help you check correctness.
- Implement locally to test for correctness, then run on cluster for benchmarking (accuracy and speed).
- Leaderboard for some assignments (minimize perplexity given training budget).

AI policy

- Coding agents can solve all the assignments, but you won't learn anything.
- AI can be tremendously useful for answering questions and tutoring.
- You must use our provided AGENTS.md file, which asks the AI to be pedagogically-minded.
- Please read our AI policy guide.

Compute

- Thanks to Modal for providing compute. ��
- Please read the guide on how to access and use the compute.

```
def course_syllabus():
    basics()              # Assignment 1: tokenization, model architecture, training
    systems()             # Assignment 2: kernels, parallelism, inference
    scaling_laws()        # Assignment 3: scaling laws
    data()                # Assignment 4: evaluation, curation, transformation, filtering, deduplication, mixing
    alignment()           # Assignment 5: RLHF, RL algorithms, RL systems
```

Remember it's all about **efficiency**:
- Resources: data + hardware (compute, memory, communication bandwidth)
- How do you train the best model given a fixed set of resources?

Today, we are compute-constrained, so design decisions will reflect squeezing the most out of given hardware.
- Systems: clearly about efficiency
- Tokenization: working with raw bytes is elegant, but compute-inefficient with today's model architectures
- Model architecture: many changes motivated by reducing memory or FLOPs (e.g., sharing KV caches, sliding window attention)
- Data filtering: avoid wasting precious compute updating on bad / irrelevant data
- Scaling laws: use less compute on smaller models to do hyperparameter tuning

Tomorrow, we will become data-constrained...

```
class Tokenizer(ABC):
    """Abstract interface for a tokenizer."""
    def encode(self, string: str) -> list[int]:
        raise NotImplementedError

    def decode(self, indices: list[int]) -> str:
        raise NotImplementedError
```

```
def basics ():
```

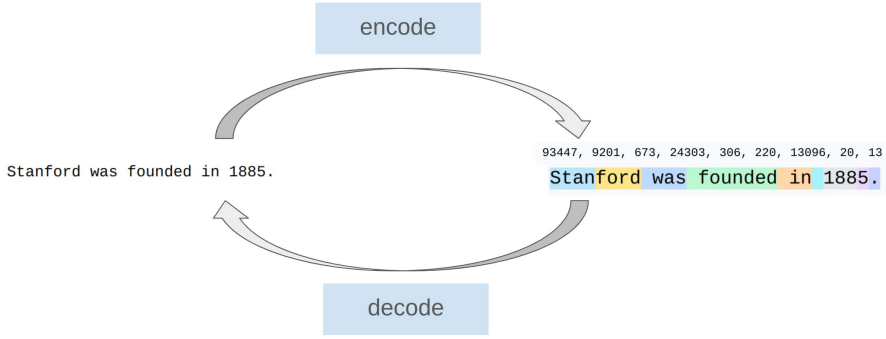
Goal: be able to train a basic language model

Components: tokenization, model architecture, training

Tokenization

What are the atoms that the model operates on?

Formally: a tokenizer converts between raw inputs (bytes) and sequences of integers (tokens)



Popular tokenizer: **Byte-Pair Encoding** (BPE) [Sennrich+ 2015]

Intuition: break input into frequently-occuring chunks

Efficiency lens

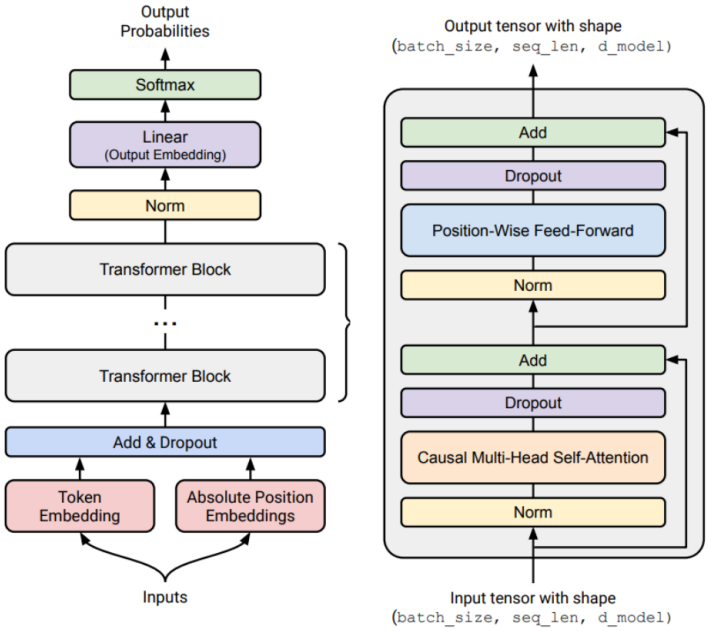
- Reduce context length (1000 bytes → ~250 tokens)
- Adaptive computation (more modeling capacity on interesting parts of input)

The dream: tokenizer-free model architectures, which operate directly on bytes [Xue+ 2021][Yu+ 2023] [Pagnoni+ 2024][Deiseroth+ 2024][Hwang+ 2025]

These are promising, but have not yet been scaled up to the frontier.

Model architecture

Starting point: original Transformer [Vaswani+ 2017]



Refinements:

- Activation functions: ReLU, SwiGLU [Shazeer 2020]
- Positional encodings: sinusoidal, RoPE [Su+ 2021]
- Normalization: LayerNorm, RMSNorm, QK norm, pre-norm versus post-norm [Ba+ 2016][Zhang+ 2019] [Dehghani+ 2023][Xiong+ 2020]
- Attention: full, sparse/local attention, group-query attention (GQA), multi-head latent attention (MLA) [Child+ 2019][Ainslie+ 2023][DeepSeek-AI+ 2024]
- Recurrence/state-space models/linear attention: Mamba, Gated DeltaNet [Katharopoulos+ 2020][Dao+ 2024]

[Yang+ 2024][Lahoti+ 2026]

- MLP: dense, mixture of experts [Shazeer+ 2017][Fedus+ 2021]
- Shape (hidden dimension, depth, number of heads, number of experts)

Training

How do you set the parameters of the model?

- Loss function (e.g., multi-token prediction) [Gloeckle+ 2024][DeepSeek-AI+ 2024]
- Optimizer (e.g., AdamW, SOAP, Muon) [Kingma+ 2014][Loshchilov+ 2017][Vyas+ 2024][Keller 2024]
- Initialization scale (e.g., Xavier init, muP) [Glorot+ 2010][Yang+ 2022]
- Learning rate schedule (e.g., cosine, WSD) [Loshchilov+ 2016][Hu+ 2024]
- Regularization (e.g., dropout, weight decay)
- Batch size (e.g., critical batch size) [McCandlish+ 2018]
- MoE specific: load balancing (e.g., aux-free) [Wang+ 2024][DeepSeek-AI+ 2024]

Assignment 1 (basics)

[GitHub][PDF]

- Implement BPE tokenizer
- Implement Transformer, cross-entropy loss, AdamW optimizer, training loop
- Do resource accounting
- Train on TinyStories and OpenWebText
- Leaderboard: minimize OpenWebText perplexity given 45 minutes on a B200 [last year's leaderboard]

High-level principle: everything is about balancing the following:

- Expressivity (can represent complex dependencies in the data)
- Stability (keep parameter and gradient norms in goldilocks zone)
- Efficiency (runs fast on hardware, both training and inference)

```
def systems():
```

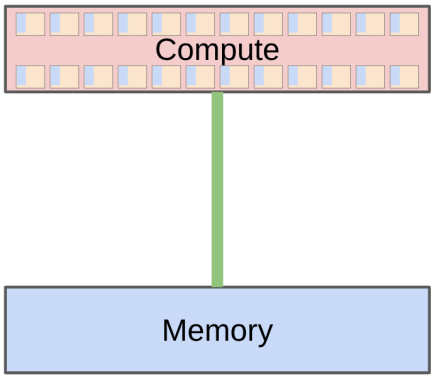
Goal: squeeze the most out of the hardware (GPU or TPU)

Components: kernels, parallelism, inference

Basics

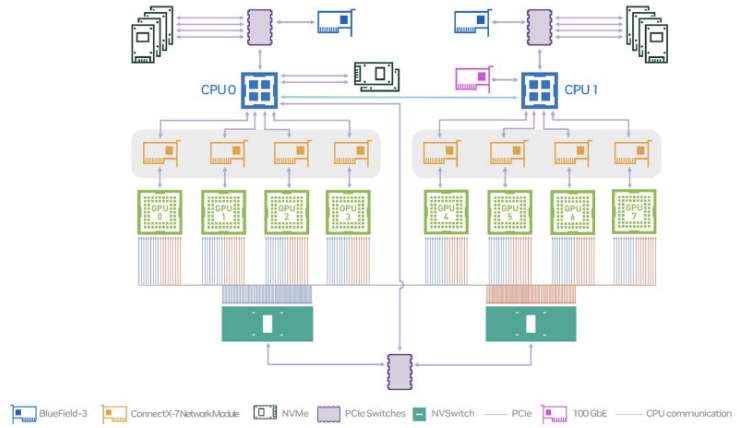
- Resource accounting: memory and compute characteristics of a model

```
total_flops = 6 * 70e9 * 1e12 # Training 70B parameters on 1T tokens = 4.2e23 FLOPs
```



- Model parameters must be moved from memory (HBM) to the compute (SMs)
- Example: B200 can perform 2.25 PFLOP/sec (bf16) with 8TB/sec memory bandwidth
- Roofline analysis: understand whether we're compute-bound or memory-bound
- Benchmarking and profiling (nsight): see what happens in practice

DGX B200:



Kernels

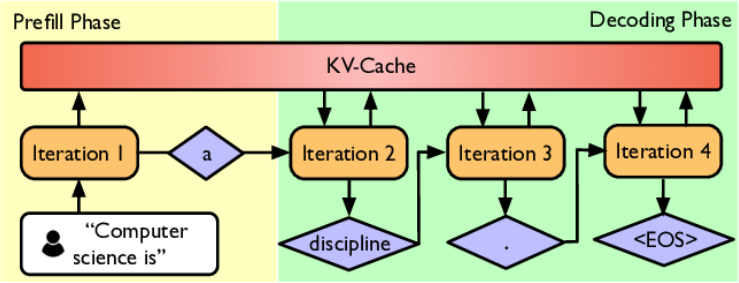
- Kernel is a function that runs on GPU
- When using PyTorch, each primitive operation launches a standard kernel
- Can write custom kernels to make GPUs go brrr
- Principle: organize computation to minimize data movement
- Naive: read HBM; compute A; write HBM; read HBM; compute B; write HBM
- Fused: read HBM; compute A and B; write HBM
- Strategies: operator fusion (matmul + activation), tiling (FlashAttention)
- Warp divergence, memory coalescing, bank conflicts, occupancy, bulk-async memory transfers
- Write kernels in CUDA/**Triton**/CUTLASS/ThunderKittens

Parallelism

- What if we have 1024 GPUs?
- Data movement between GPUs is even slower, but same 'minimize data movement' principle holds
- Use classic collective operations (e.g., gather, reduce, all-reduce)
- Shard memory (parameters, activations, gradients, optimizer states) across GPUs
- How to split computation: {data,tensor,pipeline,sequence,expert} parallelism

Inference

Goal: generate tokens given a prompt (needed to actually use models!)
Inference is also needed for reinforcement learning, test-time compute, evaluation
Two phases: prefill and decode



- Prefill (similar to training): tokens are given, can process all at once (compute-bound)
- Decode: need to generate one token at a time (memory-bound)

Methods to speed up decoding:

- Use cheaper model (via model pruning, quantization, distillation)
- Speculative decoding: use a cheaper "draft" model to generate multiple tokens, then use the full model to score in parallel (exact decoding!)
- Systems optimizations: fused kernels, continuous batching

Assignment 2 (systems)

[GitHub][PDF from Spring 2025]

- Implement a fused RMSNorm kernel in Triton
- Implement distributed data parallel training
- Implement optimizer state sharding

- Benchmark and profile the implementations

Recommended book: How to Scale Your Model
- Nicely lays out how to approach systems for LLMs conceptually
- From Google, so it foregrounds TPUs, but high-level concepts are similar

```
def scaling_laws():
```

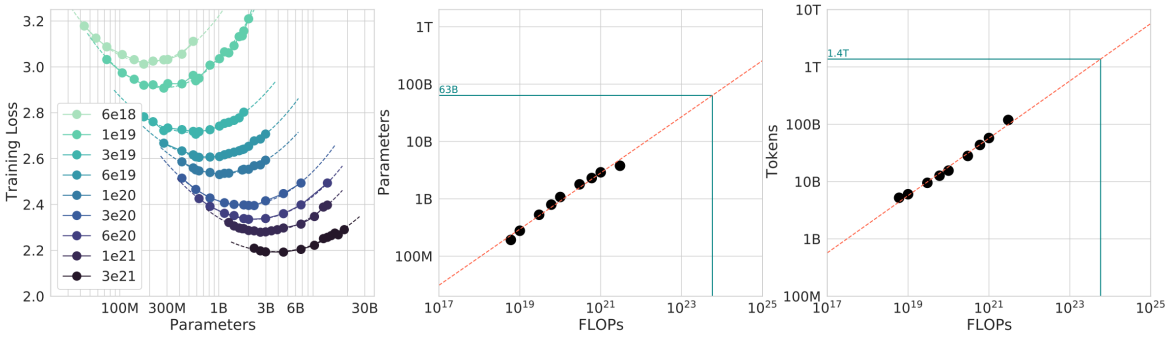
Setting: if you had 1e25 FLOPs of compute, what hyperparameters would you use to train a good model?
Too expensive to do hyperparameter tuning at full scale!

Key conceptual shift: instead of a single scale, think of a **scaling recipe** (FLOPs → hyperparameters)
For a scaling recipe:
- Run experiments to compute the loss at various smaller scales (e.g., up to 1e24 FLOPs)
- Fit a scaling law to predict the loss of the scaling recipe at the target scale (e.g., 1e25 FLOPs)

Now you can:
1. Optimize the scaling recipe targeting a larger scale using smaller scale experiments
2. Predict the loss at the target scale before actually running the experiment!

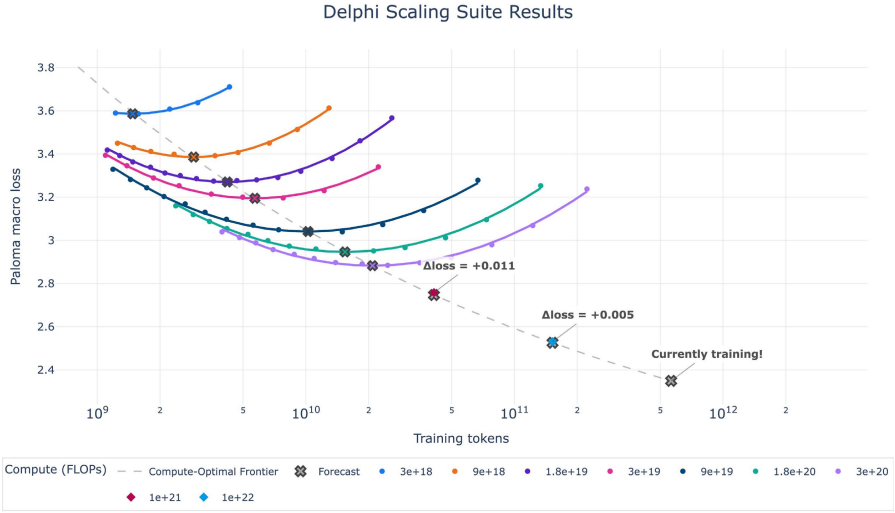
Scaling laws don't happen automatically, they require careful construction of a scaling recipe.
Parameterize the model in a way to get **hyperparameter transfer** [Yang+ 2022]
Predictability is at least as important as optimality!

Question: given a FLOPs budget (C = 6 N D), use a bigger model (N) or train on more tokens (D)?
Classic compute-optimal scaling laws: [Kaplan+ 2020][Hoffmann+ 2022]
- ISOFLOP curves: for multiple small FLOPs budgets, find optimal N
- Then fit a scaling law to extrapolate to large FLOPs budgets



TL;DR: D = 20 N is roughly optimal (e.g., 70B parameter model should be trained on ~1.4T tokens)
Caveat: this doesn't take into account inference costs (want a smaller model)

Live example from Marin [post]



Should be done training this week, should see how well we match the preregistered loss!

Assignment 3 (scaling laws)

[GitHub][PDF from Spring 2025]

- We define a training API (hyperparameters → loss) based on previous runs
  - Submit "training jobs" (under a FLOPs budget) and gather data points
  - Fit scaling laws to the data points
  - Submit extrapolated hyperparameters and loss predictions
  - Leaderboard: minimize loss given FLOPs budget

```
def data():
```

Question: What capabilities do we want the model to have?
Multilingual? Good at conversation? Agentic coding capabilities?

Evaluation

What is the purpose of evaluation?

1. Internal: guide model development (smoothness across scales, relative performance matters)
2. External: measure absolute quality of a real use case (ecological validity matters)

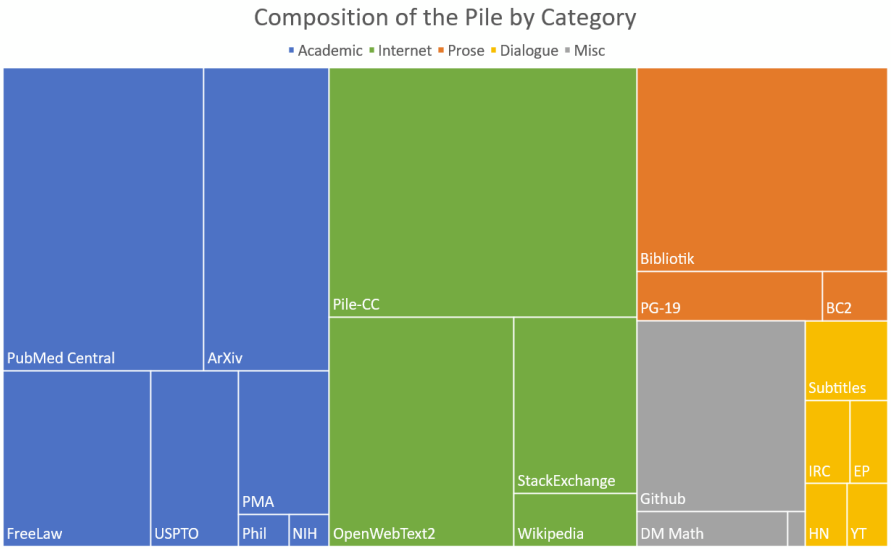
Examples of evaluations:

1. Perplexity: ideally run on private documents not on Internet (avoid contamination)
2. Advanced use cases: GPQA, HLE, SWE-Bench, Terminal-Bench

LMs are general purpose, require a diverse set of evaluations!

Data curation

- Data does not just fall from the sky.
- Sources: webpages crawled from the Internet, books, arXiv papers, GitHub code, etc.



- Appeal to fair use to train on copyright data? [Henderson+ 2023]
- Might have to license data (e.g., Google with Reddit data) [article]
- Raw data is HTML, PDF, directories (not text), requires processing

Data processing

- Transformation: convert HTML/PDF to text (extract main content)
- Filtering: keep high quality data, remove harmful content (via classifiers)
- Deduplication: save compute, avoid memorization; use Bloom filters or MinHash
- Data mixing: how much to upweight/downweight each source? [Liu+ 2024][Chen+ 2026]
- Rewriting / synthetic data: use LM to augment real data, more similar to downstream tasks [Maini+ 2024]

Types of data:

- Pretraining data: large and diverse
- Mid-training data: high quality, including long-context
- Post-training data: supervised fine-tuning (conversations, agentic traces with tool calling)

Assignment 4 (data)

[GitHub][PDF from Spring 2025]

- Convert Common Crawl HTML to text
- Train classifiers to filter for quality and harmful content
- Deduplication using MinHash
- Leaderboard: minimize perplexity given token budget

def alignment ():

So far, we have trained a model on full supervision (predict the next token).
Now that the model should be reasonable, we can improve it further from **weak supervision**.
Why weak supervision? When it is easier to critique than to generate.

Basic template:

1. Generate responses from the model.
2. Score responses with a {human, verifier, LM judge}.
3. Update the model to prefer better responses.

Algorithms:

- Proximal Policy Optimization (PPO) from reinforcement learning [Schulman+ 2017][Ouyang+ 2022]
- Direct Policy Optimization (DPO): for preference data, simpler [Rafailov+ 2023]
- Group Relative Preference Optimization (GRPO): remove value function [Shao+ 2024]

Challenges:

- RL algorithms are unstable and hard to tune
- At scale, this requires a lot of new infrastructure (inference with async rollouts)
- Constantly trading off systems efficiency and on-policyness

Assignment 5 (alignment)

[GitHub][PDF from Spring 2025]

- Implement Direct Preference Optimization (DPO)
- Implement Group Relative Preference Optimization (GRPO)

##
Tokenization

def tokenization ():

This unit was inspired by Andrej Karpathy's video on tokenization; check it out! [video]

```
intro_to_tokenization()
tokenization_examples()
character_tokenizer()
byte_tokenizer()
word_tokenizer()
bpe_tokenizer()
```

Summary:

- Tokenizer: strings ↔ tokens (indices)
  - Character-based, byte-based, word-based tokenization are highly suboptimal
  - BPE is an effective heuristic that is data-driven
  - Tokenization is a separate step, maybe one day do it end-to-end from bytes...

But whatever solution needs to satisfy:

1. Model (e.g., Transformer) should operate on chunks (abstractions) of the sequence (text, video, DNA, etc.)
2. Chunks should be variable (allocate more model capacity to interesting chunks)

```python


class CharacterTokenizer(Tokenizer):
    """Represent a string as a sequence of Unicode code points."""
    def encode(self, string: str) -> list[int]:
        return list(map(ord, string))

    def decode(self, indices: list[int]) -> str:
        return "".join(map(chr, indices))


class ByteTokenizer(Tokenizer):
    """Represent a string as a sequence of bytes."""
    def encode(self, string: str) -> list[int]:
        string_bytes = string.encode("utf-8")
        indices = list(map(int, string_bytes))
        return indices

    def decode(self, indices: list[int]) -> str:
        string_bytes = bytes(indices)
        string = string_bytes.decode("utf-8")
        return string


def merge(indices: list[int], pair: tuple[int, int], new_index: int) -> list[int]:
    """Return `indices`, but with all instances of `pair` replaced with `new_index`."""
    new_indices = []
    i = 0
    while i < len(indices):
        if i + 1 < len(indices) and indices[i] == pair[0] and indices[i + 1] == pair[1]:
            new_indices.append(new_index)
            i += 2
        else:
            new_indices.append(indices[i])
            i += 1
    return new_indices


@dataclass(frozen=True)
class BPETokenizerParams:
    """All you need to specify a BPETokenizer."""
    vocab: dict[int, bytes]     # index -> bytes
    merges: dict[tuple[int, int], int]  # index1,index2 -> new_index


class BPETokenizer(Tokenizer):
    """BPE tokenizer given a set of merges and a vocabulary."""
    def __init__(self, params: BPETokenizerParams):
        self.params = params

    def encode(self, string: str) -> list[int]:
        indices = list(map(int, string.encode("utf-8")))
        # Note: this is a very slow implementation
        for pair, new_index in self.params.merges.items():
            indices = merge(indices, pair, new_index)
        return indices


```

```
        def decode(self, indices: list[int]) -> str:
    bytes_list = list(map(self.params.vocab.get, indices))
    string = b"".join(bytes_list).decode("utf-8")
        return string


def get_compression_ratio(string: str, indices: list[int]) -> float:
    """Given `string` that has been tokenized into `indices`, return the number of UTF-8 bytes per token."""
  num_bytes = len(bytes(string, encoding="utf-8"))
  num_tokens = len(indices)
    return num_bytes / num_tokens


def get_gpt5_tokenizer():
    # Code: https://github.com/openai/tiktoken
    return tiktoken.get_encoding("o200k_base")


def intro_to_tokenization():
```

Raw text is generally represented as Unicode strings.

```
  string = "Hello, 🌍! 你好!"
```

A language model places a probability distribution over sequences of tokens (usually represented by integer indices).

```
  indices = [15496, 11, 995, 0]
```

So we need a procedure that *encodes* strings into tokens.
We also need a procedure that *decodes* tokens back into strings.
A Tokenizer is a class that implements the encode and decode methods.

```
def tokenization_examples():
```

To get a feel for how tokenizers work, play with this [interactive site]

Observations

- A word and its preceding space are part of the same token (e.g., " world").
- A word at the beginning and in the middle are represented differently (e.g., "hello hello").
- Numbers are tokenized into every few digits.

Here's the GPT-5 tokenizer from OpenAI (tiktoken) in action.

```
  tokenizer = get_gpt5_tokenizer()
  string = "Hello, 🌍! 你好!"
```

Check that encode() and decode() roundtrip:

```
  indices = tokenizer.encode(string)
  reconstructed_string = tokenizer.decode(indices)
    assert string == reconstructed_string
```

Compression ratio: number of bytes per token

```
  compression_ratio = get_compression_ratio(string, indices)
```

The larger the compression ratio, the shorter the sequence (good since attention is quadratic in sequence length).
One could increase compression ratio by increasing **vocabulary size** (number of possible token values increases), leading to sparsity.

```
  vocabulary_size = tokenizer.n_vocab
```

Let's take a look at the actual vocabulary: [vocab]

```
  output_tokenizer(tokenizer, "var/gpt5_tokenizer_vocab.txt")
```

```
616
617
618 def output_tokenizer(tokenizer, path: str):
619     """Write out the vocabulary of `tokenizer` to `path`, one per line."""
620     if not os.path.exists(path):
621         vocab = [b.decode("utf-8", errors="replace") for b in tokenizer.token_byte_values()]
622         with open(path, "w") as f:
623             for token in vocab:
624                 f.write(token + "\n")
625
626
627 def character_tokenizer():
628     A Unicode string is a sequence of Unicode characters.
629     Each character can be converted into a code point (integer) via `ord`.
630     assert ord("a") == 97
631 assert ord("🙃") == 127757
632     It can be converted back via `chr`.
633     assert chr(97) == "a"
634 assert chr(127757) == "🙃"
635
636     Now let's build a `Tokenizer` and make sure it round-trips:
637 tokenizer = CharacterTokenizer()
638 string = "Hello, 🙃! 你好!"
639 indices = tokenizer.encode(string)                  # call ord
640 reconstructed_string = tokenizer.decode(indices)    # call chr
641     assert string == reconstructed_string
642
643     There are approximately 150K Unicode characters. [Wikipedia]
644 vocabulary_size = max(indices) + 1  # This is a lower bound
645     Problem 1: this is a very large vocabulary.
646 Problem 2: many characters are quite rare (e.g., 🙃), which is inefficient use of the vocabulary.
647 compression_ratio = get_compression_ratio(string, indices)
648     This tokenizer is the worst of both worlds (large vocabulary, low compression ratio).
649
650
651 def byte_tokenizer():
652     Unicode strings can be represented as a sequence of bytes, which can be represented by integers between 0
        and 255.
653     The most common Unicode encoding is [UTF-8]
654
655     Some Unicode characters are represented by one byte:
656     assert bytes("a", encoding="utf-8") == b"a"
657     Others take multiple bytes:
658 assert bytes("🙃", encoding="utf-8") == b"\xf0\x9f\x8c\x8d"
659
660     Now let's build a `Tokenizer` and make sure it round-trips:
661 tokenizer = ByteTokenizer()
662 string = "Hello, 🙃! 你好!"
663 indices = tokenizer.encode(string)
664 reconstructed_string = tokenizer.decode(indices)
665     assert string == reconstructed_string
666
667     The vocabulary is nice and small: a byte can represent 256 values.
668 vocabulary_size = 256
669     What about the compression rate?
670 compression_ratio = get_compression_ratio(string, indices)
671     assert compression_ratio == 1
672     The compression ratio is terrible, which means the sequences will be too long.
```

Given that the context length of a Transformer is limited (since attention is quadratic), this is not looking great...

```
def word_tokenizer():
```

Another approach (closer to what was done classically in NLP) is to split strings into words.

```
string = "I'll say supercalifragilisticexpialidocious!"

chunks = regex.findall(r"\w+|." , string)
```

This regular expression keeps all alphanumeric characters together (words).

To turn this into a `Tokenizer`, we need to map these chunks into integers.
Then, we can build a mapping from each chunk into an integer.

What's good: each token is meaningful (since humans invented words).

```
vocabulary_size = "Number of distinct chunks in the training data"
compression_ratio = get_compression_ratio(string, chunks)
```

Compression ratio is good, but vocabulary size can be huge.

Moreover:

- Many words are rare and the model won't learn much about them.
- This doesn't obviously provide a fixed vocabulary size.
- New words we haven't seen during training get a special UNK token, which is ugly and can mess up perplexity calculations.

```
def bpe_tokenizer():
```

Byte Pair Encoding (BPE)

The BPE algorithm was introduced by Philip Gage in 1994 for data compression. [article]
It was adapted to NLP for neural machine translation. [Sennrich+ 2015]
(Previously, papers had been using word-based tokenization.)
BPE was then used by GPT-2. [Radford+ 2019]

Basic idea: *train* the tokenizer on raw text to construct a vocabulary tailored to the data.
Intuition: common sequences of bytes are represented by a single token, rare sequences are represented by many tokens.

Sketch: start with each byte as a token, and successively merge the most common pair of adjacent tokens.

Training the tokenizer

```
string = "the cat in the hat"
params = train_bpe(string, num_merges=3)
```

Using the tokenizer

Now, given a new text, we can encode it.

```
tokenizer = BPETokenizer(params)
string = "the quick brown fox"
indices = tokenizer.encode(string)
reconstructed_string = tokenizer.decode(indices)
    assert string == reconstructed_string
```

In Assignment 1, you will go beyond this in the following ways:

- encode() currently loops over all merges. Only loop over merges that matter.
- Detect and preserve special tokens (e.g., <|endoftext|>).
- Use pre-tokenization (e.g., the GPT-2 tokenizer regex).
- Try to make the implementation as fast as possible.

```
728
729 def train_bpe(string: str, num_merges: int) -> BPETokenizerParams:
730     Start with the list of bytes of string.
731     indices = list(map(int, string.encode("utf-8")))
732     merges: dict[tuple[int, int], int] = {}  # index1, index2 => merged index
733     vocab: dict[int, bytes] = {x: bytes([x]) for x in range(256)}  # index -> bytes
734
735     for i in range(num_merges):
736         # Count the number of occurrences of each pair of tokens
737         counts = count_adjacent_pairs(indices)
738
739         # Find the most common pair
740         pair = max(counts, key=counts.get)
741
742         # Merge that pair
743         new_index = 256 + i
744         merges[pair] = new_index
745         vocab[new_index] = vocab[pair[0]] + vocab[pair[1]]
746         indices = merge(indices, pair, new_index)
747
748     compression_ratio = get_compression_ratio(string, indices)
749
750     return BPETokenizerParams(vocab=vocab, merges=merges)
751
752
753 def count_adjacent_pairs(indices: list[int]) -> dict[tuple[int, int], int]:
754     """Return a dictionary mapping each adjacent pair of tokens in `indices` to the number of times it occurs."""
755     counts = defaultdict(int)
756     for index1, index2 in zip(indices, indices[1:]):
757         counts[(index1, index2)] += 1
758     return counts
759
760
761 if __name__ == "__main__":
762     main()
```